

REPORTE TAREA 2

ALGORITMOS Y COMPLEJIDAD

«¿Quien ganará? Greedys vs Dinámico vs Bruto.»

Javier Gana

25 de junio de 2026

00:56

1. Entorno experimental

Procesador: AMD Ryzen 5 5600H with Radeon Graphics (3.30 GHz)

Memoria RAM: 16,0 GB (13,9 GB utilizable)

Sistema Operativo: Windows 11 utilizando WSL

Entorno de desarrollo: Visual Studio Code

Lenguajes de programación y compilador: Compilador g++ (Ubuntu 13.3.0-6ubuntu2 24.04.1) 13.3.0 y Python 3.14 (64-bit)

Para los casos pequeños se usaron {3, 5, 8} cantidad de animes, con un limite de 4 capítulos cada uno. Esas cantidades fueron escogidas para garantizar que el algoritmo de fuerza bruta no tardara demasiado en finalizar su ejecución.

Para casos medianos se usaron {20, 40, 80} cantidad de animes.

Para casos grandes se usaron {100, 150, 200} cantidad de animes.

2. Resultados y Análisis

2.1. Análisis de resultados: Greedy vs Fuerza Bruta

2.1.1. Medición de tiempo de ejecución

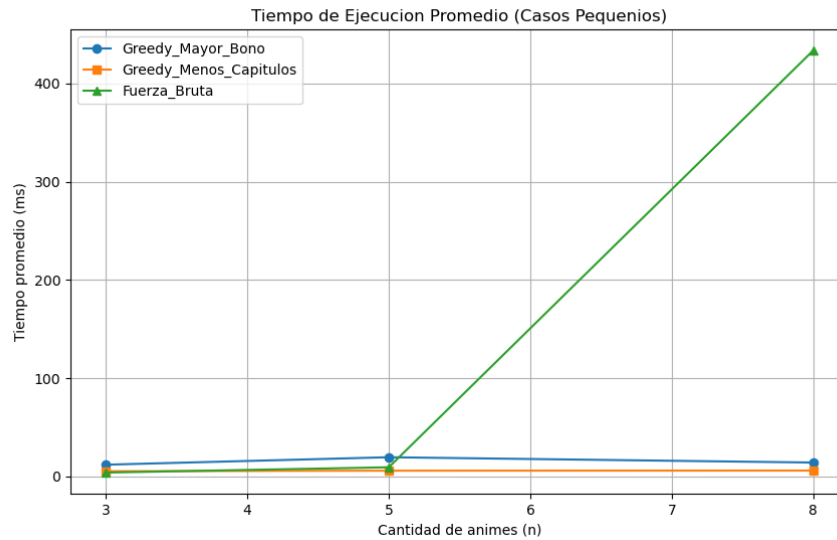


Figura 1: Gráfico del tiempo de ejecución promedio para casos pequeños.

Este gráfico se centra en visualizar de buena manera la comparación entre el algoritmo de Fuerza Bruta con los algoritmos Greedy. La figura deja en evidencia la baja eficiencia del algoritmo de Fuerza Bruta, ya que mientras los Greedy mantienen un tiempo de ejecución pequeño ante tan baja cantidad de anime, el algoritmo de Fuerza bruta incrementa su tiempo de ejecución de gran manera cuando revisa el caso de 8 anime.

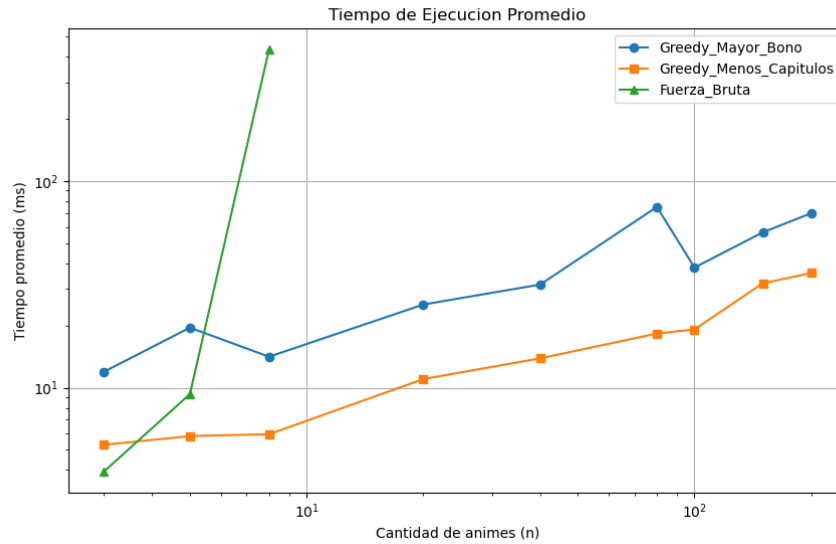


Figura 2: Gráfico del tiempo de ejecución promedio para todos los casos.

Puede verse como ambos algoritmos Greedy crecen a ritmo similar mientras que, siguiendo con la idea anterior, se puede notar como al algoritmo de Fuerza Bruta incrementó su tiempo de ejecución de manera exagerada al aumentar la cantidad de anime, dándose la situación de que aún en grandes casos, los algoritmos Greedy tardaban menos que el de Fuerza Bruta con pocos anime.

También podemos notar en este gráfico que inicialmente el algoritmo de Fuerza Bruta es el más rápido de los 3, esto puede deberse a que este algoritmo va directo al punto ; mientras que ambos Greedy comienzan haciendo comparaciones para ver por donde empezar, lo cual en casos muy muy pequeños provoca que tarden más.

2.1.2. Medición de satisfacción máxima (Calidad de la solución)

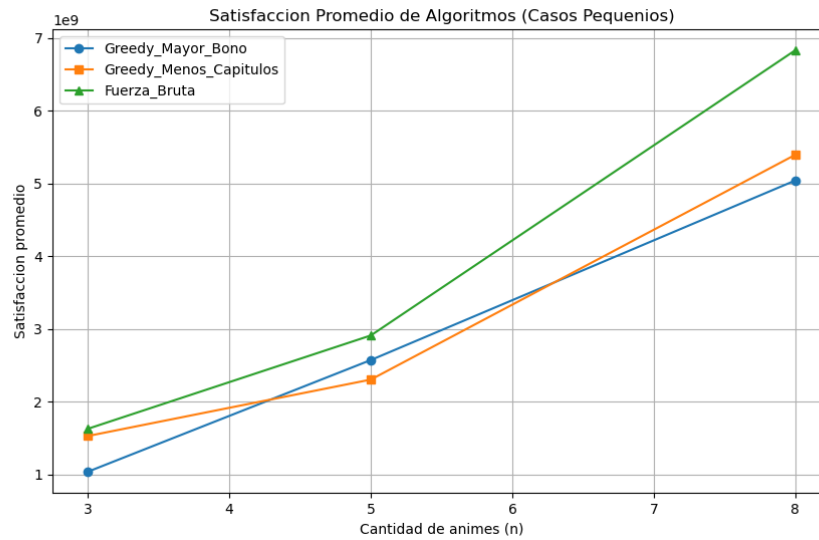


Figura 3: Gráfico de la satisfacción promedio obtenida para casos pequeños.

Respecto a la calidad de los resultados, el gráfico muestra lo esperado teóricamente: la Fuerza Bruta siempre alcanza el punto más alto de satisfacción. Al explorar todo el árbol de decisiones, este algoritmo garantiza encontrar la solución óptima absoluta al problema.

Por otro lado, los algoritmos Greedy toman decisiones locales y definitivas sin evaluar el impacto a largo plazo, lo que inherentemente los lleva a soluciones subóptimas y a quedarse por debajo de la cota superior que es Fuerza Bruta. A pesar de esto, se puede observar que logran alcanzar puntajes bastante cercanos al óptimo en una fracción del tiempo.

Al comparar las dos heurísticas Greedy con el algoritmo de Fuerza Bruta, podemos concluir que los Greedy sacrifican la precisión absoluta a cambio de un tiempo de ejecución viable.

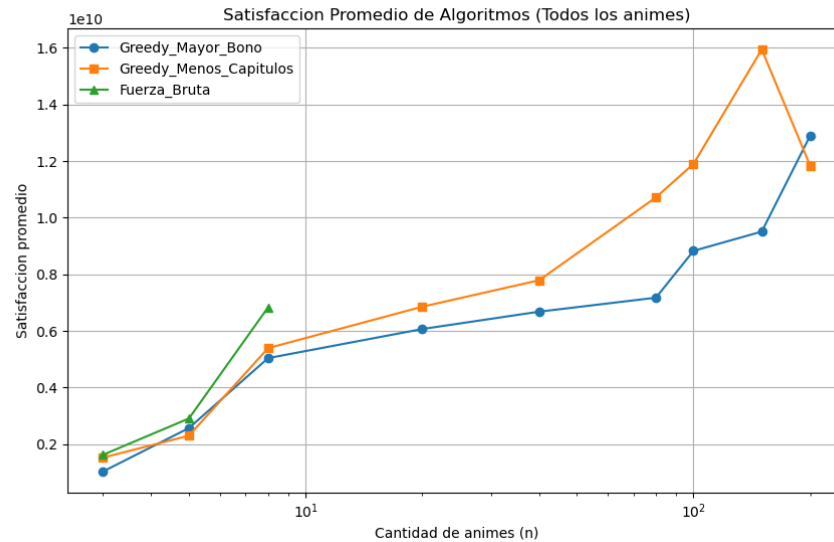


Figura 4: Gráfico de la satisfacción promedio obtenida para todos los casos.

A partir del gráfico podemos deducir que el algoritmo Greedy_Mayor_Bono tiende a fallar más que Greedy_Menos_Capitulos, ya que este último tiende a obtener mayores valores de satisfacción.

3. Conclusiones

Respecto a los resultados obtenidos, el gráfico de tiempo deja claro que Fuerza Bruta crece demasiado rápido, lo cual lo hace inviable para casos grandes, a diferencia de los algoritmos Greedy que son muy veloces y escalan sin problemas. En cuanto a la satisfacción, era de esperarse que Fuerza Bruta consiga siempre el puntaje máximo. Sin embargo, aunque los Greedy no llegan a ese óptimo, logran resultados bastante cercanos en una fracción del tiempo. Finalmente, al comparar ambos algoritmos Greedy, puede notarse claramente en el último gráfico que Greedy_Menos_Capitulos es el ganador, ya que logra una mayor satisfacción que el otro Greedy en casi todos los tamaños probados.

Respecto al espacio, no se mostró acá ya que en todas las mediciones dio 0, esto es debido al trabajo interno del sistema operativo, que se encarga de asignar el espacio. Este programa no llega a completar el espacio que se le asignó de base, así que nunca pide memoria extra lo cual dificulta mucho la medición del espacio utilizado.